

02-614 Week 1

String Algorithms

Aidan Jan

August 27, 2026

Strings

A string is a data structure that represents a sequence of letters drawn from an alphabet. Some examples:

- “abcdefg”
- “hello”
- “ACCGTACGG...”

This course covers algorithms used for processing strings.

Exact Matching

One of the first questions we might ask is where a string exists in another string, if it exists. Formally, given a string T and a string P , report all places in T where P occurs.

Naïve Algorithm

```
for every pos i in T:
    for every pos j in P:
        check if T[i + j] = P[j]
```

This algorithm essentially checks every position in the string T to see if P begins at that position. This has a running time of $O(|T| \times |P|)$, which is slow. Ideally, we want an algorithm with $O(|T| + |P|) = O(|T|)$ instead, since that is theoretically the fastest time possible.

The Z Algorithm

This algorithm does exact matching in linear time.

Define Z -values. $Z_i(P) :=$ the length of the longest substring starting at i in P that matches a prefix of P . (e.g., a repeated substring in P).

```
def Zmatch(T, P):
    S = concat(P, '$', T)
    print all i for which Z_i = |P|
```

Essentially, if we knew all the Z_i 's, then finding an exact matching is trivial. The only issue now is, how do we compute the Z_i 's?

Computing Z_i Values

Assume we have computed $Z_2, \dots, Z_{k-1}$, and ℓ and r , where $\ell, r :=$ the left and right edges of the rightmost non-zero z_i values computed so far. The section of z -values between ℓ and r are called the z -box. From here we have a few cases:

1. Case $k > r$: The Z -box ends before index k . If this case occurs, we have to brute-force compute Z_k
2. Case $k \leq r$: The Z -box ends after index k . We know that $\ell < k$ since the start of a z -box must be within the part we already computed. Therefore ℓ is at most $k - 1$. We know for a fact that the characters in a Z -box matches the characters at the beginning of the string. Therefore, since k is in our Z -box, there must exist a k' in the same box at the beginning of the string which we have already calculated. Now, define β as the section of the string between k and r . Now we have two subcases:
 - (a) Subcase $Z_{k'} < \beta$: We know that $Z_k = Z_{k'}$.
 - (b) Subcase $Z_{k'} \geq \beta$: If this is true, then $Z_{k'}$ extends past the end of the Z -box. Therefore, we skip to r , and compute from that point using brute force.

Knuth-Morris-Pratt (KMP) Algorithm (With Z-Values)

Similar to the Z -algorithm, we are going to define an auxiliary data structure that gives us the answer to the problem. For this solution, we will define an `spm[i]` array, where

`spm[i]` := length of the longest substring of P that ends at i and that matches a prefix of P , and where $P[i + 1] \neq P[\text{spm}[i] + 1]$.

If we know the entire `spm[i]` array, then if we reach a mismatch, we don't have to check every character like the string matching, since the `spm` array

Runtime

Every time we get a mismatch, we shift our window by at least 1. This bounds the number of mismatches at $|T|$. Since the KMP algorithm only checks each matched character once, the maximum number of matches is also bounded by $|T|$. Therefore, there must be less than $2|T|$ iterations since every character must be a match or a mismatch. The runtime is $O(|T|)$.

Computing `spm[i]`

Recall the definition of the Z -value, which is Z_i is the length of the longest substring of P that starts at i and matches a prefix of P . This is very similar to `spm` values.

Now, define $f(i)$ as the right end of the Z -box starting at i . Note that it is possible for multiple Z -boxes end at the same position. For this, we define $g(j)$ as the smallest i where $f(i) = j$. In other words, $g(j)$ picks the longest Z -box that ends at a given position.

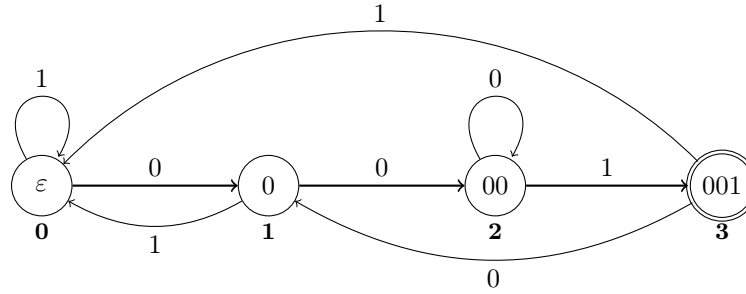
Theorem:

For each i , `spm[i]` = $Z_{g(i)}$ if $Z_{g(i)} > 0$ otherwise `spm[i]` = 0. In other words, if no Z -box ends at i , then the `spm` value for that position is 0.

Knuth-Morris-Pratt (KMP) Algorithm (Without Z-Values)

The traditional KMP algorithm is defined without Z -values, but instead, using DFAs. (Deterministic Finite Automaton)

The first step is to generate a DFA for pattern matching for P . Suppose we do; the process of building it will be described later. Then, use T as instructions to traverse the DFA for matches. Following is an example for $P = \text{"001"}$ and $T = \text{"1100010"}$.



Steps in order: $0 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 3$ (Print match!) $\rightarrow 1$

However, in this case, the DFA might be hard to build. This is because the number of edges in the DFA scales on the size of the alphabet. To circumvent this, the KMP DFA only uses two types of edges, “Match” and “Mismatch”. For a binary alphabet, the graph would look basically exactly the same, but in a case where it is not, we need some more definitions.

Let $\text{memo}[k]$ be the length of the longest proper suffix of $P[1, \dots, k]$, and also the definition of the mismatch edges. Note that this is very similar to the spm definition.

Pseudocode for KMP

Here, we first assume we have the $\text{memo}[k]$.

```

def KMP(P, T):
    j = 0 // our current state in the DFA
    for every position i = range(1, |T|) do:
        while j > 0 and T[i] != P[j + 1] do:
            j = memo[j]
        if T[i] == P[j + 1] then:
            j++
        if j == |P|:
            print match
            j = memo[j]
  
```